

Parallel Incremental Maintenance of TPC-H Query 3 with Apache Flink

YUN Hanxu

Student ID: 21286712

Program: MSc in Information Technology

Course: CSIT6910 Independent Project

Supervisor: Prof. Ke Yi

Email: hyunac@connect.ust.hk

Code: <https://github.com/silverpioup/Self-Project>

Abstract

I implemented continuous maintenance of TPC-H Query 3 over insertions and deletions using Apache Flink. The implementation specializes the live-tuple algorithm described by Wang and Yi and demonstrated in Cquirrel to the foreign-key chain `lineitem -> orders -> customer`. It stores base tuples, reverse indexes, live-state information, and aggregate revenue in Flink managed state. Only a transition of a live root lineitem changes the query result. Orders and lineitems are partitioned across keyed workers, while customer updates are replicated to the order shards. Revenue is represented with exact fixed-point integers, and the official ordered Top-10 is maintained after each input update.

Correctness was checked independently with SQLite. The verifier reconstructed the complete Flink result and compared all groups and every emitted Top-10 change. All checks passed for hand-written corner cases, a 156,600-update synthetic stream, a small TPC-H fixture, and a 1,140,816-update TPC-H SF0.1 FIFO stream. A repeated local experiment used parallelism 1, 2, 4, and 8 and confirmed that every configured worker was active. Parallelism 2 achieved the highest mean processing throughput, 895,123.76 updates/s.

1. Background and Related Work

Continuous analytical applications evaluate a registered query while the base relations change. Re-running a multi-table query after each tuple update repeats joins and aggregation work that is unrelated to the changed tuple. Incremental view maintenance instead updates a data structure for the current database and enumerates only the difference between the old and new answers.

Wang and Yi study this problem for acyclic primary-key/foreign-key joins. Their algorithm maintains live tuples and supports insertions and deletions to any relation. Its amortized update bound is expressed using the enclosure of an update sequence. FIFO streams are an important low-enclosure case. The paper further shows how the method supports all TPC-H queries and reports an implementation on Flink.

Cquirrel presents the corresponding continuous-query system. It accepts an initially empty database followed by a Flink DataStream of insertions and deletions, and uses delta enumeration as the output model. Its Q3 example maintains indexes for customer, orders, lineitem, and aggregation. My implementation applies that pattern to Q3 and adds exact correctness checks and parallelism instrumentation. It is limited to Q3 and does not include Cquirrel's general query compiler.

TPC-H Q3 was selected because it is the three-table example identified in the project briefing. It combines an acyclic foreign-key join with selections, grouped SUM, ordering, and a Top-10 limit.

2. Query and Algorithm

The maintained query is:

```
SELECT l_orderkey,
```

```

        SUM(l_extendedprice * (1 - l_discount)) AS revenue,
        o_orderdate,
        o_shippriority
FROM customer, orders, lineitem
WHERE c_mktsegment = 'BUILDING'
      AND c_custkey = o_custkey
      AND l_orderkey = o_orderkey
      AND o_orderdate < DATE '1995-03-15'
      AND l_shipdate > DATE '1995-03-15'
GROUP BY l_orderkey, o_orderdate, o_shippriority
ORDER BY revenue DESC, o_orderdate
LIMIT 10;

```

Under the paper's edge direction, the foreign-key DAG is `lineitem -> orders -> customer`; `lineitem` is the root and `customer` is the leaf. A customer tuple is live when it passes the BUILDING selection. An order is live when it passes the order-date selection and references a live customer. A lineitem is live when it passes the ship-date selection and references a live order. Thus a live root tuple corresponds to exactly one qualifying Q3 join result.

For each non-leaf tuple the implementation stores a child-match count and a live flag. Q3's graph is a chain, so this count is zero or one. Inserting, deleting, or changing the live state of a tuple propagates only through reverse indexes to its direct parents. When a lineitem changes from non-live to live, its fixed-point revenue is added to its group; the reverse transition subtracts the same amount. This is the Q3 specialization of bottom-up live-tuple maintenance. No intermediate customer-orders or orders-lineitem join is materialized.

The group key is `(orderkey, orderdate, shippriority)`. After applying all deltas belonging to one input sequence, the global stage updates a sorted ranking with revenue descending, followed by order date and order key for deterministic ties. It emits a complete Top-10 only when that ranking changes.

Prices are parsed as cents and discounts as hundredths. The contribution `extendedprice * (100 - discount) / 100` is retained internally as a signed 64-bit integer in units of 0.0001 dollars. SQLite uses the identical integer formula. Consequently, validation does not use an epsilon or floating-point tolerance.

3. Implementation Details

The system uses Java 11-compatible bytecode, JDK 17, Apache Flink 1.19.1, and Maven. A single file source preserves the bounded stream order. A process function assigns a global sequence number and checkpoints that number as operator state.

Orders and lineitems are routed by `orderkey mod p`, where `p` is the requested parallelism. Customer updates are copied to all shards because one customer's orders can span several shards. Flink's default hash partitioning does not guarantee that small integer shard identifiers use all subtasks. `RoutingPlan` therefore uses Flink's key-group assignment to choose one partition key that maps to each physical subtask. Unit tests verify the one-to-one mapping for parallelism 1, 2, 4, 8, and 16.

The parallel `CquirrelQ3ShardFunction` maintains customers, orders, orders-by-customer, lineitems, lineitems-by-order, live flags, and child-match counts in keyed `MapState`. A strict update contract rejects duplicate insertions, missing-tuple deletions, malformed field counts, invalid dates, and invalid monetary values. A replacement is represented by delete followed by insert so that every delta has an unambiguous inverse.

Shard responses retain the input sequence. The global keyed function waits for all expected responses, which is especially important for replicated customer updates, then combines group deltas and applies them in sequence order. Its managed state stores current group revenue and pending sequence batches. A sorted in-memory ranking is rebuilt from managed revenue state after recovery.

Optional checkpoints use Flink's EXACTLY_ONCE mode for managed keyed and operator state. The evaluation sink prints to standard output and is not a transactional sink. Therefore the state is recoverable

exactly once, but a failure could duplicate already printed records. A production deployment would use a checkpoint-aware Kafka or filesystem sink.

The repository also contains deterministic workload generators, official DBGen RF1/RF2 conversion, SQLite verification, repeated benchmarking, unit tests, portable PowerShell runners, raw results, and this report.

4. Experimental Setup and Data Sets

Experiments were executed on Windows 10 with JDK 17.0.19, 20 logical CPUs, and a 4 GB JVM heap. Flink ran locally in one JVM. Each performance setting used one warm-up and three measured repetitions. The benchmark records two throughput values: internal processing throughput from the first sequenced update to completion of the last update, and wall-clock throughput including local Flink/JVM startup and shutdown. Latency is measured once per original update, after all shard responses and the global Top-10 update complete.

The principal data set was generated using DuckDB's TPC-H-compatible dbgen at SF0.1: 15,000 customers, 150,000 orders, and 600,572 lineitems. A deterministic FIFO generator first loads all customers and a 75,000-order active window. For each remaining order, it deletes the oldest order's lineitems and then the order, followed by the new order and its lineitems. The stream contains 1,140,816 updates: 765,572 insertions and 375,244 deletions. This workload models the FIFO update sequence discussed in the algorithm paper; it is not an audited TPC benchmark result.

Additional tests used a 13-update hand-written stream, an eight-update TPC-H fixture, and a 156,600-update synthetic stream. The hand-written stream covers late-arriving parents, lineitem deletion, order deletion and reinsertion, customer predicate changes, and group removal.

For correctness, the same stream is replayed into an independent in-memory SQLite database. At selected sequences, the verifier executes grouped Q3 SQL and compares every exact group with the state reconstructed from Flink deltas. Every Top-10 emitted by Flink is compared with the SQLite ranking at that exact sequence.

5. Experimental Results and Discussion

All recorded correctness checks passed:

Workload	p	Updates	Group snapshots	Top-10 changes	Result
Hand-written	1, 2, 4, 8	13	13 each	6 each	PASS
TPC-H fixture FIFO	4	8	8	1	PASS
Synthetic	8	156,600	16	351	PASS
TPC-H SF0.1 FIFO	8	1,140,816	12	141	PASS

Every comparison reported zero group mismatches. The SF0.1 test validates more than a final snapshot: it checks 12 complete states and all 141 points at which Flink emitted a changed Top-10.

The repeated SF0.1 measurements were:

p	Active	Processing updates/s	Std. dev.	Wall updates/s	Mean us	P50 us	P95 us	P99 us
1	1	654,465.24	68,597.79	93,109.04	7,547.01	6,340	14,736.67	27,103.33
2	2	895,123.76	138,713.22	96,765.30	6,543.48	5,233.33	14,846.67	31,353.33
4	4	828,483.05	35,911.26	96,157.05	5,275.98	3,196.67	16,320.00	44,766.67
8	8	756,174.26	12,483.74	94,893.49	10,567.00	6,680	34,033.33	73,646.67

The active-worker count equalled the configured parallelism in every measured run. Worker-level record counters also showed non-zero work on every subtask, so these are genuine parallel executions rather than repeated single-worker runs.

Parallelism 2 produced the highest mean internal throughput, 36.8% above parallelism 1. Parallelism 4 was 26.6% above parallelism 1 and had the lowest mean and median latency. Parallelism 8 remained 15.5% above parallelism 1 in throughput but increased tail latency. The non-monotonic result is expected from this design: customer updates are replicated to every shard, routing work grows with p , and exact sequence ordering plus global group/ranking maintenance is centralized. The experiment therefore demonstrates useful parallelism but not linear scalability.

Wall-clock throughput is nearly flat because each bounded run spends about ten seconds starting and closing the local Flink runtime. Internal processing throughput better isolates the algorithm, while wall-clock throughput describes the experience of running the supplied command. Both values are retained to avoid selecting only the more favorable measure.

6. Future Directions

The next step would be to replace customer replication with a customer-to-shard subscription index and evaluate the operators on a multi-node Flink cluster. A long-running Kafka source would remove bounded-job startup from steady-state measurements. Failure-injection tests with a transactional sink would verify end-to-end exactly-once behavior. Finally, a compiler could derive the live-state indexes and propagation rules for a broader class of SPJA queries, including general DAGs where assertion keys are required.

References

1. Q. Wang and K. Yi, "Maintaining Acyclic Foreign-Key Joins under Updates," *Proceedings of ACM SIGMOD*, 2020, doi:10.1145/3318464.3380586.
2. Q. Wang et al., "Cquirrel: Continuous Query Processing over Acyclic Relational Schemas," *Proceedings of the VLDB Endowment*, 2021.
3. Transaction Processing Performance Council, *TPC Benchmark H Standard Specification*, current version.
4. Apache Flink, *Flink 1.19 Documentation*.

Appendix A: Meeting Minutes

Minutes of the 1st Project Meeting

Date: Wednesday, June 10, 2026

Time: 4:00 pm-4:22 pm

Place: Online meeting

Present: Prof. Ke Yi; YUN Hanxu; other CSIT6910 students

Apology: None

Note-taker: YUN Hanxu

1. Approval of minutes

This was the first project meeting, so there were no previous minutes to approve.

2. Discussion items

The supervisor introduced TPC-H and the project directions. For the Flink project, the query should involve at least three tables; Q3 was identified as the standard minimum example. The work should generate insertions and deletions, implement the Cquirrel/AJU update algorithm with a Flink process function and maintained state, emit affected groups, verify snapshots with a conventional database, and then evaluate throughput and parallelism.

3. Meeting adjournment and next meeting

The meeting adjourned at approximately 4:22 pm. The next meeting would review the selected query, update representation, and state design.

Minutes of the 2nd Project Meeting

Date: Thursday, July 2, 2026

Time: 4:00 pm-4:30 pm

Place: Online meeting

Present: Prof. Ke Yi; YUN Hanxu

Apology: None

Note-taker: YUN Hanxu

1. Approval of minutes

The first meeting's requirements and the selection of TPC-H Q3 were reviewed.

2. Discussion items

The Q3 predicates, foreign-key chain, group key, and fixed-point revenue expression were confirmed. The update representation was defined for all three relations. The planned state included primary-key maps, customer-to-order and order-to-lineitem indexes, live flags, child-match counts, and group revenue. SQLite was selected as the independent correctness baseline.

3. Meeting adjournment and next meeting

The meeting adjourned at 4:30 pm. The next meeting would review the runnable Flink implementation and sample correctness results.

Minutes of the 3rd Project Meeting

Date: Friday, July 24, 2026

Time: 9:00 am-9:30 am

Place: Online meeting

Present: Prof. Ke Yi; YUN Hanxu

Apology: None

Note-taker: YUN Hanxu

1. Approval of minutes

The query, update format, state indexes, and SQLite verification plan from the second meeting were reviewed.

2. Discussion items

The Java/Flink job and changed-group output were demonstrated. Insertions, deletions, late-arriving parents, replacement operations, and removal of zero-revenue groups were tested. The verification procedure was extended to reconstruct the complete Flink state and compare every group and emitted Top-10 with SQLite. The remaining work was to run larger TPC-H FIFO data, verify worker activity, repeat the parallel experiment, and finish the report.

3. Meeting adjournment and next meeting

The meeting adjourned at 9:30 am. The next meeting would review the final correctness evidence, parallel measurements, limitations, and report.

Minutes of the 4th and Final Project Meeting

Date: Thursday, August 6, 2026

Time: 4:00 pm-4:30 pm

Place: Online meeting

Present: Prof. Ke Yi; YUN Hanxu

Apology: None

Note-taker: YUN Hanxu

1. Approval of minutes

The implementation and verification actions recorded in the third meeting were reviewed.

2. Discussion items

The completed implementation and evaluation were reviewed. The discussion covered the Q3 live-tuple propagation rules, exact fixed-point aggregation, managed state and checkpoint recovery, the deterministic TPC-H SF0.1 FIFO stream, and the independent SQLite comparisons. The parallelism 1, 2, 4, and 8 results were examined together with the active-worker records. The limitations of the local experiment were recorded: customer updates are replicated, the ordered Top-10 stage is single-threaded, and the console sink is not transactional. The final report, experiment records, run instructions, and GitHub repository were checked for consistency.

3. Meeting adjournment and next meeting

The meeting adjourned at 4:30 pm. This was the fourth and final project meeting, and no further meeting was scheduled. The final code, experiment records, and report were prepared for submission after the meeting-record corrections.